

Task Management API - Stage 3: Payload Validation & Record Creation (POST)

Overview

This repository houses a lightweight RESTful Task Management API built using Node.js and Express in VS Code on Windows as part of the FlyRank Internship Backend Engineering Track (Week 3).

Stage 3 introduces record creation capabilities (POST /tasks) with incoming body payload parsing via `express.json()`, input validation rules, dynamic primary key (ID) allocation, and strict HTTP status responses (201 Created and 400 Bad Request).

Technical Stack & Environment

- **Runtime:** Node.js (v24.x)
- **Framework:** Express.js
- **Middleware:** `express.json()` (Deep Packet Inspection / Body Parser)
- **Data Layer:** In-Memory Volatile Array (RAM Cache)
- **Environment:** VS Code Integrated Terminal (Windows PowerShell pwsh)
- **Verification Tool:** `curl.exe` (Native Windows Binary)
- **Version Control:** Git

API Catalog & Route Architecture

Method	Endpoint	HTTP Success	HTTP Error	Sysadmin / IT Analogy
GET	/	200 OK	—	IIS Web Site Root Banner / Discovery
GET	/health	200 OK	—	Load Balancer ICMP Heartbeat Ping
GET	/tasks	200 OK	—	Querying service list (Get-Service)
GET	/tasks/:id	200 OK	404 Not Found	Querying

				Event Viewer ID (Get-WinEvent -InstanceId)
POST	/tasks	201 Created	400 Bad Request	Helpdesk Intake Form Verification & Dynamic Primary Key Allocation

Architectural Breakdown: Stage 3 Concepts

1. express.json() Ingress Middleware (Deep Packet Inspection / DPI)

- **Concept:** By default, HTTP POST payloads arrive as raw TCP byte streams. The `express.json()` middleware intercepts inbound packets with `Content-Type: application/json`, parses the JSON stream into a JavaScript object, and attaches it to `req.body`.

2. Defensive Validation Gate (HTTP 400 Bad Request)

- **Rule:** System endpoints **never trust incoming client data**. If an incoming payload lacks a title property, supplies a non-string, or provides a blank string `""`, the server halts processing early and responds with HTTP 400 Bad Request.
- **Sysadmin Analogy:** Equivalent to PowerShell parameter validation rules `[Parameter(Mandatory=$true)]` rejecting empty arguments before script execution starts.

3. Dynamic ID Generation & HTTP 201 Created

- **ID Allocation Logic:** `tasks.length > 0 ? Math.max(...tasks.map(t => t.id)) + 1 : 1` dynamically assigns the next available integer key.
- **Status Code 201 Created:** Unlike 200 OK (general request success), 201 Created acts as an official system receipt confirming a new resource was instantiated in system state.

Verification & Terminal Execution Log

1. Launch Server Daemon

In your **left terminal tab**:

```
node server.js
```

2. Valid Record Creation (POST /tasks)

In your **right terminal tab**:

```
curl.exe -i -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{"title":"Buy milk"}'
```

- **Response Header:** HTTP/1.1 201 Created
- **Response Body:** {"id":4,"title":"Buy milk","done":false}

3. State Persistence Check (GET /tasks)

```
curl.exe -i http://localhost:3000/tasks
```

- **Response Header:** HTTP/1.1 200 OK
- **Response Body:** Array containing 4 tasks including id: 4.

4. Input Validation Firewall Test (HTTP 400 Bad Request)

```
curl.exe -i -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{}'
```

- **Response Header:** HTTP/1.1 400 Bad Request
- **Response Body:** {"error":"Title is required and must be a non-empty string"}

Version Control Snapshot

```
git add .
```

```
git commit -m "stage 3: create with validation"
```